

Andmed API-st

11IT · Veebirakendused · JavaScripti moodul, 2. nädal · konspekt

Kust andmed tulevad?

Eelmisel nädalal kirjutasid ülesanded ise lehele. Päril rakendustes nii ei käi. Ilmateade, uudised, bussiajad, sinu sõnumid: kõik see tuleb serverist, mõnest arvutist kuskil internetis. Leht ise hoiab ainult kujundust ja küsib andmed siis, kui neid vaja läheb.

Mõttele restoranile. Sa ei lähe ise kööki pannide juurde, vaid vaatad menüüd ja tellid. Menüü on kokkulepe: seal on kirjas, mida küsida saab ja mis nimega. Just seda tähendab **API**. Sinu kood on klient, server on köök ja tellimus jõuab lauda vastusena.

JSON: andmed teksti kujul

Ava brauseris aadress `pokeapi.co/api/v2/pokemon/pikachu`. Ekraanile tuleb pikk tekst. Esimesel korral näeb see hirmus välja, aga vaata seda natuke aega ja tunned selle ära: see on peaaegu sama asi, mis eelmise nädala objekt.

```
{
  "name": "pikachu",
  "height": 4,
  "weight": 60
}
```

Selline kuju ongi **JSON**. Ainus erinevus JavaScripti objektist on see, et võtmed on jutumärkides. Ja kuna kuju on kokku lepitud, oskab seda lugeda iga programm, ükskõik mis keeles see on kirjutatud.

fetch: kolm rida, mis toovad andmed

Andmete küsimiseks on käsk **fetch**. Retsept on alati sama, õpi see pähe:

```
async function lae() {
  const vastus = await fetch("https://pokeapi.co/api/v2/pokemon/pikachu");
  const andmed = await vastus.json();
  console.log(andmed.name);
}

lae();
```

Rida haaval: esimene rida läheb internetti ja toob vastuse, teine teeb vastusest tavalise objekti ja kolmas kasutab seda nii, nagu sa juba oskad.

Uus sõna on **await**. See tähendab „oota ära“. Päring internetti võtab aega, umbes nagu tellimus köögist, ja ilma ootamiseta jookseks programm tühja taldrikuga edasi. Kui kuskil koodis on **await**, peab funktsiooni ette olema kirjutatud **async**. Need kaks käivad alati koos.

Kui näed konsoolis Promise {<pending>}: see tähendab „vastus tuleb, aga veel mitte“. Peaaegu alati on põhjus see, et **await** jäi kirjutamata.

Andmed lehele

Kui andmed on käes, on ülejäänud juba eelmisest nädalast tuttav. JSONPlaceholder on harjutamiseks mõeldud API, mis annab näidisülesandeid:

```
async function laeYlesanded() {
  const vastus = await fetch("https://jsonplaceholder.typicode.com/todos?_limit=5");
  const ylesanded = await vastus.json();

  for (const y of ylesanded) {
    lisaRida(y.title);
  }
}
```

Aadressi lõpus olev `?_limit=5` ütleb serverile, et viiest ülesandest piisab. Muidu tuleb neid kakssada ja leht on üks pikk müür.

Vaata päringut oma silmaga

Vajuta F12 ja vali vahekaart **Network**. Värskena lehte ja vajuta oma nuppu: iga päring ilmub sinna ritta. Klõpsa sellel ja vaata staatusekoodi.

Kood	Mida see ütleb
200	Kõik korras, andmed on käes.
404	Sellist aadressi ei ole. Kontrolli, kas kirjutasid selle õigesti.
500	Serveris läks midagi katki. Sinu koodis ei pruugi viga ollagi.

Proovi meelega ära rikkuda: kirjuta aadressi `todos` asemel `todoss` ja vaata, mis Network näitab. Vea nägemine on väärt rohkem kui selle vältimine.

Sõnastik

Mõiste	Lihtsalt öeldes
server	arvuti internetis, mis hoiab andmeid
API	kokkulepe, kuidas serverilt andmeid küsida
JSON	tekstikuju, milles andmed internetis liiguvad
fetch	käsk, millega sinu kood andmeid küsib
await	„oota, kuni vastus kohale jõuab“

Kontrolli ennast

- Mis on API? Seleta see restorani näitega.
- Mille poolest erineb JSON eelmise nädala objektist?
- Mida teeb await ja miks on selle kõrval vaja async-i?
- Mida tähendab konsoolis Promise {<pending>}?
- Mida ütleb staatusekood 404?